Document number:	P1295R0
Date:	2018-10-07
Audience:	Library Evolution Working Group
Reply-to:	*Tomasz Kamiński <tomaszkam at gmail dot com>*

Spaceship library update

1. Introduction

This paper proposed a set of minor usability tweaks for the library portion of the tree-way comparison operator: comparison category types.

2. Revision history

2.1. Revision 0

Initial revision.

3. Motivation and Scope

3.1. Equality for the comparison categories

Currently the comparison category types defined in the standard (`weak_equality`, `strong_equality`, `partial_ordering`, ...) are not providing the equality operator for comparing they values. Instead the suggested way to inspect the comparison result is to compare it against the zero constant (`0`).

The above mechanism is working effectively, for situations when we want to check if the category object `res` represent one of the following result:

- equal/equivalent: `res == 0`,
- non-equal/non-equivalent: `res != 0`,
- less: `res < 0`,
- less-or-equal: `res <= 0`,
- greater: `res > 0`,
- greater-or-equal: `res >= 0`.

However, if we want to check if the `partial_ordering` object `part` is representing unordered result (`partial_order::unordered`), we are forced to check if it does not represent any of the above result, by using one of the following:

```
!(part < 0) && !(part == 0) && !(part > 0)
!(part <= 0) && !(part >= 0)
```

Both of above solution are less efficient that possible library implementation, that have access to the underlying representation of the object.

Secondly, we are missing an efficient way to check if two comparison results are having the same value, in situation if nor of them has a runtime know value. For example imagine that we want to implement an unanimous comparison predicate (see P0100: Comparison in C++) — the objects are considered to be less, if all corresponding fields are less (same for greater or equal).

To illustrate, lets consider implementation for of above predicate for class `A` containing members `x`, `y`, `z`:

```
std::partial_ordering unanimous_compare(A const& lhs, A const& rhs)
{
   auto first_result = lhs.x <=> rhs.y;
   if (auto res = lhs.x <=> rhs.y; !is_same(first_result, res))
     return std::partial_ordering::unordered;
   if (auto res = lhs.z <=> rhs.z; !is_same(first_result, res))
     return std::partial_ordering::unordered;
   return first_result == 0 ? first_result : std::partial_ordering::unordered;
}
```

And `is_same` for the existing comparison categories:

```
bool is_same(std::weak_equality lhs, std::weak_equality rhs)
// handles std::strong_equality via conversion
{
   if (lhs == 0)
     return rhs == 0;

   // lhs != 0
   return rhs != 0;
}

bool is_same(std::partial_ordering lhs, std::partial_ordering rhs)
{
    if (lhs == 0)
      return rhs == 0;

    if (lhs < 0)
      return rhs < 0;

    if (lhs > 0)
      return rhs > 0;

    // lhs is unordered
    return !(rhs <= 0) && !(rhs >= 0);
}

bool is_same(std::weak_ordering lhs, std::weak_ordering rhs)
// optimization for std::weak_ordering and std::strong_ordering
{
    if (lhs == 0)
      return rhs == 0;

    if (lhs < 0)
      return rhs < 0;

    // lhs > 0
    return rhs glt; 0;
}
```

Again above functions may be implemented more effectively in library, that can just compare underlying representation.

To address both of above issues, we propose to make comparison categories equality comparable, thus allowing unordered values to be checked by `part == std::partial_ordering::unordered` and replacing the whole `is_same` implementation by simple invocation of comparison operator (`lhs == rhs`).

3.2. `common_type` for comparison categories

For the comparison category types `C...`, the standard currently provides two independent traits to produce their common type:

- the standard library mechanism: `std::common_type_t<C...>`,
- specialized `std::common_comparison_category_t<C...>`.

In the authors opinion, regardless if the programmer will use existing general purpose trait (`common_type`) or the specialized version (`common_comparison_category`), they should get the same result.

Currently above is not held for the following combination of the comparison categories:

- `strong_equality` and `partial_ordering`,
- `strong_equality` and `weak_ordering`.

For the above types, `common_comparison_category` is `weak_equality`, while `common_type` does not have nested `type` typedef.

To fix above we are proposing that `common_type<C...>::type` shall be the same as `common_comparison_category<C...>::type` in case when all types in pack `C` are comparison category types. This may be achieved by providing following specializations of `common_type`:

```
template<> struct common_type<strong_equality, partial_ordering> { using type = weak_equality; };
template<> struct common_type<partial_ordering, strong_equality> { using type = weak_equality; };
template<> struct common_type<strong_equality, weak_ordering> { using type = weak_equality; };
template<> struct common_type<weak_ordering, strong_equality> { using type = weak_equality; };
```

Finally, proposed `common_type` extension, is required guarantee that `EqualityComparableWith` concept would be satisfied for above categories, after equality operations will be introduced.

3.3. Header for `compare_3way`

Per author's input during Albuquerque both the `compare_3way` and `lexicographical_compare_3way` functions were moved from the `compare` header to the `algorithm`. While, moving `lexicographical_compare_3way` was right decision, as it is working on iterator/ranges, moving `compare_3way` may be an mistake.

The `compare_3way` is a function designed for handling comparison in generic code, regardless if argument type defines `operator<=>` or set of two-ways comparison operators. This makes it a fundamental building block for implementing comparison operators in case of generic components, like `std::optional`. As consequence of current placement of this function, implementation of such component would be required to bring content of the `algorithm` header.

To avoid above overhead we are proposing the move `compare_3way` function back to `compare` header, as it was originally proposed.

4. Design Decisions

4.1. No ordering for comparison categories

This paper purposely does not propose to define an orderings for the comparison categories. While defining some kind of ordering would be technically possible (as we have small set of elements), we have not found motivating use cases for such feature, in addition we reckon that any definition of ordering would not intuitive, especially when you consider how `unordered` should be ordered respective to `less`, `equivalent` and `greater` values.

4.2. Placement of `common_type` specialization

This paper is specifically avoiding providing an explicit specialization of the `common_type` in the `compare` header (like it is done for the `chrono`), to avoid potential bloat of the header - instead the programmers are still required to include `type_traits` to use an `common_type` for common comparison categories. [Note: They can still use dedicated lightweight `common_comparison_category` trait.]

This decision is motivated by the fact, that inclusion of the `compare` is required both to declare `operator<=>` and use relation operators (`<=>`, `<`, `>`, ...) on class, an thus we expect this header to me include by majority of translation units.

5. Proposed Wording

TBD.

6. Acknowledgements

Herb Sutter for discussion and feedback on changes proposed in this document.

7. References

1. Herb Sutter, Jens Maurer, Walter E. Brown, "Consistent comparison" (P0515R3, http://wg21.link/p0515r3)
2. Lawrence Crowl, "Comparison in C++" (P0100R2, http://wg21.link/p0100r2)
3. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4762, https://wg21.link/n4762)